# Operational Forensic Audit and Strategic Architecture Report: Phase 4 Library Hardening (The Stela Autopoietic Ecosystem)

## 1. Executive Strategic Analysis: The Autopoietic Turn

The contemporary software engineering landscape is currently undergoing a paradigm shift of unprecedented magnitude, transitioning from static, human-maintained repositories to dynamic, agentic ecosystems.[1] The Stela platform, as evidenced by the forensic artifacts provided in the repository dump and the associated "AI Agent Constitution Review," stands at the vanguard of this transition. This report provides an exhaustive operational forensic audit of the platform's current state, specifically validating the execution of Dispatch Packet DP-OPS-0026, and analyzes the systemic implications of the newly established Policy of Truth (PoT) framework.[1]

The analysis proceeds from the fundamental understanding that Stela serves not merely as a codebase but as an implementation of *autopoietic systems theory* within a software development lifecycle.[1] Autopoiesis, a concept originally derived from biological cybernetics, describes systems capable of reproducing and maintaining themselves within a topological boundary. In the context of Stela, this translates to a "Project Factory" architecture where the system's operating kernel (ops/) stands decoupled from its variable payloads (projects/), creating a clean separation of concerns analogous to the "kernel-user space" division in monolithic operating systems. This distinction remains critical because, unlike allopoietic systems which produce something other than themselves (like a car assembly line producing cars), an autopoietic system's primary product serves as its own continued organization.

### 1.1 The Theoretical Framework of Autopoietic Software

To rigorously evaluate the success of the recent Phase 3 transition (DP-OPS-0026), one must first deconstruct the theoretical underpinnings that govern Stela's architecture. Stela represents a stark departure from traditional "Citizen Models" of AI interaction toward a "Mechanistic Doctrine".[1] This transition functions not merely as a change in tooling but as a fundamental reorientation of the "Social Contract" between the human operator and the artificial agent.

#### 1.1.1 The Mechanistic Doctrine vs. The Citizen Model

The "Citizen Model" of AI interaction anthropomorphizes artificial agents, treating them as

trusted collaborators capable of discerning intent, understanding nuance, and "doing the right thing" amidst ambiguity. This model relies heavily on social contracts, implied context, and high-trust environments. While effective in human-centric teams where shared cultural context bridges gaps in documentation, the Citizen Model introduces catastrophic risks in agentic systems due to "Epistemic Drift".[1] Epistemic Drift occurs when a corruption in the rule-defining mechanism—such as a template, a linter, or a prompt instruction—propagates exponentially through the system, creating a cascade of architectural divergence.

In contrast, the "Mechanistic Doctrine," which Stela formally adopted in Phase 3, posits that the system must physically and deterministically prevent the agent from performing unauthorized actions.[1] It rejects trust in favor of verification. Under this doctrine, an agent does not receive a request to follow a style guide; the system undergoes engineering to crash if the style guide suffers violation. This shift proves critical because, unlike human operators who may fatigue, rationalize shortcuts, or "bend the rules" for expediency, autonomous agents operate with relentless speed and absolute literalism.[1] If a vulnerability exists in the "Project Factory" (specifically the ops/bin/project binary), every project instantiated by that factory inherits the vulnerability, leading to systemic fragility.

The successful execution of DP-OPS-0026 enshrined this doctrine within the Preamble of the PoT.md file, explicitly stating: "The system prevents action that violates policy. It does not request compliance".[1] This constitutes a historic milestone for the repository, moving it from a state of "aspirational governance" to "deterministic enforcement."

### 1.1.2 From System Injection to Project Election

Phase 3 stands operationally defined by the architectural pivot from "System Injection" to "Project Election".[1] In earlier iterations of the Stela platform, the system likely injected a global context into every agent session. While this ensured that agents held access to all available tools, it led to "Context Haze"—a degradation of AI reasoning capabilities caused by the saturation of irrelevant information. This phenomenon mirrors the "information overload" described in knowledge management theories, where the absence of a Single Source of Truth (SSOT) leads to cognitive paralysis.

The new autopoietic model mandates that the global system (ops/) remains sterile, containing only the absolute minimum tools required for construction. The capabilities—Agents, Tasks, and Skills—reside in the "Library Trinity" (docs/library/) and load into the runtime environment only when explicitly "elected" by a project's STELA.md configuration file. This reduces token overhead and sharpens agent focus, but it introduces a new dependency: **Referential Integrity**. If a project elects a capability that does not exist (a "Dead Pointer"), the autopoietic process fails. Thus, the verification of DP-OPS-0026 centers on whether the system successfully implemented the linters and guards necessary to enforce this integrity, ensuring that the "machine specification" remains valid at all times.

# 2. Forensic Audit of Repository State (HEAD 7bc5050de)

This section presents a forensic audit of the repository state (HEAD 7bc5050de) to determine if the execution of DP-OPS-0026 met its strategic objectives. The analysis relies on the "State of Play" (SoP.md) ledger, the "Policy of Truth" (PoT.md), and the file manifest provided in the repository dump.[1]

## 2.1 The Evidence of Execution: Analyzing the Ledger

The primary artifact for verification serves as SoP.md (State of Play). In the Stela governance model, SoP.md functions as the immutable history ledger; a task legally achieves "complete" status only when recorded here. This mirrors the function of a blockchain or a secure audit log in financial systems, where the record of the transaction acts as the transaction itself.[1]

**Audit Finding:** The SoP.md file contains a specific entry for 2026-02-05 - DP-OPS-0026: Phase 3 Semantic Refactoring & Structural Hygiene. This entry explicitly logs the following actions:

- **Refactoring of PoT.md nomenclature:** The log confirms the transition to "Governance Engine," "Core," "Federation," and "Tiered Jurisdiction".[1]
- **Structural Refactoring:** The log records the replacement of docs/ops/INDEX.md with an architectural docs/ops/README.md and the removal of the redundant docs/security/ surface.[1]
- **Enforcement Hardening:** The log notes updates to tools/lint/truth.sh and tools/lint/style.sh to enforce the new terminology and markdown robustness.

**Conclusion:** The presence of this detailed entry in the canonical ledger provides *prima facie* evidence that the governance requirements for recording the work met satisfaction. The system acknowledges the change, satisfying the "Equilibrium Maintenance" directive of the PoT. This adherence to the ledger prevents the "hidden drift" often seen in systems where code changes occur without corresponding governance updates.

## 2.2 Deep Dive: The Monolithic Constitution (PoT.md)

The creation and refinement of PoT.md stands as the centerpiece of Phase 3. A structural analysis of the file confirms that it effectively merges the disparate governance logic into a coherent whole, eliminating the "Scattered Truth" problem identified in the Constitution Review.[1]

### 2.2.1 Semantic Refactoring: The Retirement of the Sheriff

The most significant aspect of DP-OPS-0026 involved the semantic refactoring of

governance terms. The forensic audit of PoT.md (HEAD 7bc5050de) confirms the successful excision of the legacy "Wild West" metaphors in favor of precise, systems-theory terminology.[1]

| Legacy Term | New Term | Location Verified | Implication |
|---|---|---|---|
| **Sheriff** | **Governance Engine** | PoT.md Sec 2 | Moves from a persona-based enforcement model (discretionary) to a machine-based model (deterministic). The Governance Engine is defined as "Integrator plus Automation". |
| **Colonies** | **Federation** | PoT.md Sec 3.2 | "Colonies" implied subservience. "Federation" implies semi-autonomy within a constitutional framework, aligning with the "Tiered Jurisdiction" model where projects have local rights. |
| **Crown** | **Core** | PoT.md Sec 3.1 | "Crown" implied imperial rule. "Core" implies a central kernel or operating system, aligning with the technical reality of the ops/ directory. |
| **Multi-verse** | **Tiered** | PoT.md Sec 3 | "Multi-verse" |

| | | | |
|---|---|---|---|
| | **Jurisdiction** | | implied disconnected realities. "Tiered Jurisdiction" precisely describes the relationship between the Platform Core and the Project Federation. |

This semantic shift serves as more than cosmetic; it functions as ontological. By renaming "The Sheriff" to "The Governance Engine," the system explicitly depersonalizes enforcement. A Sheriff might grant a warning; an Engine simply executes a halt command. This aligns perfectly with the Mechanistic Doctrine's assertion that the system "prevents action" rather than "requests compliance".[1]

## 2.3 Structural Hygiene and the Filing Doctrine

The forensic audit confirms that the physical structure of the repository underwent alteration to match the "Filing Doctrine" (ops/ = Run, docs/ = Explain).[1]

### 2.3.1 Elimination of docs/security/

The file list in the dump no longer contains docs/security/README.md or the docs/security/ directory.[1] This confirms the successful pruning of redundant pointers. Security policy now stands consolidated within PoT.md Section 5.4, reducing the "Context Haze" caused by fragmented documentation. In information theory, a "pointer to a pointer" (like the old docs/security/README.md) represents inefficient encoding. By forcing security into the central Constitution, Stela ensures that every agent loads the security policy as part of the "Small Bundle" context.[1]

### 2.3.2 Refactoring of docs/ops/

The file docs/ops/INDEX.md, which previously acted as a weak pointer list, underwent replacement by docs/ops/README.md. The content of this new file strictly adheres to the architectural mandate: "The Ops Kernel is the executable companion to docs. Documentation explains and justifies; ops runs".[1] This clearly delineates the separation of concerns, ensuring that human operators look to docs/ for understanding while the system looks to ops/ for execution.

## 2.4 Linter Hardening: The Mechanistic Guarantee

Governance remains only as real as its enforcement. The audit of tools/lint/truth.sh reveals

that the semantic shift receives backing by code, not just policy.

The forbidden_spellings array in tools/lint/truth.sh now explicitly includes: "Sheriff", "Colonies", and "Crown".[1] This constitutes the "Mechanistic Doctrine" in action. The system does not merely ask contributors to use the new terms; it physically prevents the re-introduction of the old metaphors. If an agent or operator attempts to commit a file referencing "The Sheriff," the linter triggers a hard failure, halting the PR pipeline. This satisfies the PoT preamble: "The system prevents action that violates policy. It does not request compliance".[1]

---

# 3. The Library Trinity Deep Dive: Validating the Need for Phase 4

While the Core (ops/, PoT.md, SoP.md) has achieved a state of hardened, autopoietic stability ("Phase 3"), the analysis reveals a critical maturity gap in the **Library Trinity** (docs/library/agents, docs/library/tasks, docs/library/skills). The user's query identifying the need for a "deep dive" into these directories stands strategically sound and urgent. The current state of the agents, tasks, and skills represents a "capability lag" relative to the platform's "governance advance".[1]

## 3.1 The "Tethering" Gap: Untethered Agents in a Federated World

The forensic audit of the docs/library/agents/ directory (containing R-AGENT-01 through R-AGENT-06) reveals a critical flaw in the current agent architecture: **Lack of Cognitive Tethering**.

Currently, the agents exist defined as generic personas (architect, code-reviewer, integrator).[1] While ops/bin/project injects the project context at runtime, the internal self-conception of the agent—its "system prompt personality"—lacks explicit engineering to accept and prioritize that context.

**The Diagnostic:** The current agents rely on the Integrator (the runtime orchestration) to provide context. They lack the "Recursive Context Check" instruction within their own definition files. They do not explicitly know to look for STELA.md or to treat PoT.md as their binding constraints.[1]

**Risk Scenario:** In an agentic workflow, an agent that "thinks" it functions as a generic software architect will default to general best practices (e.g., "Use standard clean code") derived from its training data. However, a Stela Federation Agent must "believe" it functions as a specialized worker operating solely within the bounds of a specific project, governed by that project's STELA.md. If STELA.md mandates a specific, non-standard architectural pattern (e.g., "Use vertical slice architecture over clean architecture"), the agent must adhere to that

local law without hesitation.

### 3.2 The Integration Gap: Task-Agent-Skill Disconnect

The user correctly notes that "THE THREE CATEGORIES NEED TO WORK TOGETHER IN COMBINATIONS." Currently, the library components exist somewhat in isolation.

- **Agents** define *who* does the work.
- **Tasks** define *what* sequence of steps constitutes the work.
- **Skills** define *how* specific technical operations are performed.

A "Deep Dive" remains required to weave these into a cohesive "Instructional Graph".[1]

**Task-Agent Binding:** B-TASK-03 (Integrate Command) begins this by chaining integrator -> code-reviewer -> security-reviewer.[1] However, the forensic audit does not show evidence that the Agents themselves possess awareness of this chain. Does the Integrator agent know it holds responsibility for handing off to the Code Reviewer? Or does that logic reside solely in the task file? In a truly autopoietic system, the agents should possess "Team Awareness".

**Skill-Agent Binding:** Does R-AGENT-06 (Security Reviewer) explicitly know to invoke S-LEARN-05 (Security Protocols)? Or does it rely on generic security knowledge? The "Mechanistic Doctrine" demands that we do not trust the LLM's training data when we possess a verified Skill in the library. The agent definitions should explicitly reference the Skills they carry the expectation to wield.

### 3.3 The Isolation Requirement: "Their Own World"

The prompt specifies: "THOSE AGENTS NEED TO BELIEVE THEY OPERATE IN THEIR OWN WORLD (projects/*)." This constitutes the implementation of the "Federation" model.

- **Cognitive Containment:** When an agent undergoes instantiation for projects/alpha, it should possess no awareness of or influence by projects/beta.
- **Pathing Discipline:** Agents must undergo rigorous training (via their system prompts in docs/library/agents/) to treat ./ as the project root, not the repo root, whenever possible, or to handle the path translation flawlessly. The ops/bin/project script handles the directory switching [1], but the agent's internal reasoning must align with this relative pathing to prevent "Path Hallucination" (e.g., trying to write to ../../ops/ from inside a project).

---

## 4. Operational Analysis of the Library Trinity

To prepare for the deep dive, we analyze the current state of the three library categories based on the visible artifacts and the established patterns in the repository dump.

## 4.1 Agents (docs/library/agents/)

**Current State:**

- R-AGENT-01 (Architect) [1]: Defined as a "Senior software architect." Lists tools (Read, Grep, Glob).
- R-AGENT-02 (Code Reviewer) [1]: Defined as a "Senior code reviewer." Mandates strict checks.
- R-AGENT-06 (Security Reviewer) [1]: Focuses on OWASP Top 10.

**Deficiencies:**

- **Generic Persona:** The descriptions read like job postings for human hires ("You are a senior..."). They lack the "Mechanistic" rigidity of a machine instruction.
- **Missing Jurisdictional Preamble:** No explicit statement defines their legal standing within the Federation. They do not know they are "Federation Agents."
- **Contraction Risk:** While PoT.md bans contractions, the agent prompts themselves must undergo auditing. The user prompt requests a check for contractions like "don't," "can't," "won't" in these files.

## 4.2 Tasks (docs/library/tasks/)

**Current State:**

- B-TASK-03 (Integrate Command) [1]: Defines the workflow integrator -> code-reviewer -> security-reviewer.
- B-TASK-01 (Code Review) [1]: Defines the review checklist.

**Deficiencies:**

- **Implicit Handoffs:** The tasks describe the flow but do not define the interface between agents. How does the Architect pass context to the Coder? A standardized "Handoff Artifact" (defined in the Task) remains missing.
- **Legacy Terminology:** Tasks created prior to DP-OPS-0026 may still reference "Sheriff" or "Colonies" in their instructional text. A grep audit remains necessary.

## 4.3 Skills (docs/library/skills/)

**Current State:**

- S-LEARN-08 (Prune SoP) [1]: Contains a robust ## Provenance block. This constitutes the gold standard.
- S-LEARN-01 through 05: These legacy skills underwent refactoring in DP-OPS-0013/0014, but must undergo re-verification against the Phase 3 "Context Hazard" rules.

**Deficiencies:**

- **Context Hazard Risk:** Skills serve as "Just-In-Time" (JIT) resources. If they accidentally enter ops/lib/manifests/CONTEXT.md, they pollute the global context.[1]
- **Static Stacks:** S-LEARN-02 defines the "Stela Stack" (Next.js/FastAPI). If a project in the Federation uses a different stack (e.g., PHP/Laravel), this Skill becomes a liability. Skills must become "Dynamic" or "Polymorphic," capable of adapting to the STELA.md definition of the project.

---

# 5. Strategic Roadmap: Phase 4 Implementation Plan (DP-OPS-0027)

The recommendation proves affirmative: Yes, the time has arrived for a deep dive. The platform stands structurally sound (Phase 3 complete), but the workforce (Library) requires alignment. This must occur "WITHOUT DRIFT," meaning no new files outside the library should undergo creation, and PoT.md invariants must undergo respect.

## 5.1 Proposed Dispatch Packet: DP-OPS-0027 (Phase 4 Library Hardening)

**Objective:**

Standardize the Library Trinity (agents/, tasks/, skills/) to enforce Tiered Jurisdiction, tether all Federation logic to STELA.md, and ensure inter-category operability under the Mechanistic Doctrine.

## 5.2 Scope of Work

### A. Agent Refactoring (docs/library/agents/*)

For every agent file (R-AGENT-01 through 06):

1. **Inject Jurisdiction Preamble:** Add a standard block defining the agent's legal standing.
   - *Draft Text:* "JURISDICTION: Federation. AUTHORITY: Derived from STELA.md. CONSTRAINT: Cannot alter Core."
2. **Tethering Instruction:** Explicitly command the agent to read and cite STELA.md before taking action.
   - *Draft Text:* "PRIMARY DIRECTIVE: Align all output with the local project constitution (STELA.md). If STELA.md is silent, default to Platform Core standards."
3. **Contraction Sweep:** Remove any contractions in the prompt text (e.g., change "You're" to "You are", "Don't" to "Do not").

### B. Task Composition (docs/library/tasks/*)

For every task file (B-TASK-01 through 10):

1. **Agent Binding:** Explicitly list which R-AGENT IDs hold authorization to execute the task.
2. **Skill Binding:** Explicitly list which S-LEARN IDs should undergo loading or referencing during execution.
3. **Workflow Hygiene:** Ensure tasks do not encourage "Drift" (e.g., creating files outside of the project root).

**C. Skill Verification (docs/library/skills/*)**

1. **Context Hazard Check:** Verify no skill accidentally entered ops/lib/manifests/CONTEXT.md (re-run tools/lint/context.sh).
2. **Provenance Audit:** Ensure every skill possesses a valid ## Provenance block.
3. **Stack Agnosticism:** Review S-LEARN-02 and others for hard-coded stack assumptions. If found, wrap them in conditional logic: "IF STELA.md declares Stack: Next.js, THEN apply this skill."

### 5.3 Execution Logic

This work must undergo iterative performance.

1. **Draft:** Create the updated content locally.
2. **Verify:** Run tools/lint/truth.sh to ensure no "Sheriff/Colonies" slip in.
3. **Simulate:** Use ops/bin/project work <test-id> --agent=architect to verify the agent "wakes up" with the correct context and tethering instructions.

---

## 6. The Federation Protocol: A New Interaction Model

To satisfy the user's request "WITHOUT DRIFT," we must execute a surgical operation on the Library. We define the **Federation Protocol** as the set of rules governing agent behavior within the projects/ scope.

### 6.1 Principles of the Federation Protocol

1. **Local Supremacy:** STELA.md > Global Best Practices. The agent must respect the local law even if it contradicts its training data (e.g., "Use Tabs" vs "Use Spaces").
2. **Core Obedience:** PoT.md (Global) > STELA.md (Local) > Agent Training. The agent must never violate the Constitution (e.g., "No Secrets in Code") even if STELA.md requests it.
3. **Isolation:** Actions in Project A must never affect Project B. Agents must operate with "Cognitive Containment," treating the project directory as their entire universe.

### 6.2 Implementation in ops/bin/project

While the binary itself functions as part of the Core (and thus lies outside the Library Deep Dive scope), the usage of the binary by Agents in Tasks must reflect this protocol. The Tasks must instruct agents to use ops/bin/project commands to interact with the system, rather than

raw shell commands, ensuring that the "Governance Engine" stays in the loop.[1]

---

## 7. Implementation: Dispatch Packet DP-OPS-0027

The following Dispatch Packet is drafted in strict accordance with TASK.md standards and the analysis above. It represents the actionable output of this research report.

# DP-OPS-0027: Phase 4 Library Hardening (Federation Protocol)

## 3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/library-hardening-phase4

Base HEAD: 7bc5050de

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

## Worker must already be on the Required Work Branch listed above (no branch creation or switching). STOP if any mismatch. STOP if the Required Work Branch is missing. STOP if told to create or switch branches.

## 3.2 Required Context Load (Read Before Doing Anything)

**Worker must confirm loaded before acting:**

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - docs/library/INDEX.md
  - docs/library/SKILLS.md

# (OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN). DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

## 3.3 Scope and Safety

- **Objective:** Standardize the Library Trinity (Agents, Tasks, Skills) to enforce the Federation Protocol, tether agents to STELA.md, and eliminate contractions.
- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin/ binaries.
  - No changes to projects/ content.
  - No creation of new agents or tasks (refactor only).

### Target Files allowlist (hard gate)

- docs/library/agents/R-AGENT-01.md
- docs/library/agents/R-AGENT-02.md
- docs/library/agents/R-AGENT-03.md
- docs/library/agents/R-AGENT-04.md
- docs/library/agents/R-AGENT-05.md
- docs/library/agents/R-AGENT-06.md
- docs/library/tasks/B-TASK-01.md
- docs/library/tasks/B-TASK-02.md
- docs/library/tasks/B-TASK-03.md
- docs/library/tasks/B-TASK-04.md
- docs/library/tasks/B-TASK-05.md
- docs/library/tasks/B-TASK-06.md
- docs/library/tasks/B-TASK-07.md
- docs/library/tasks/B-TASK-08.md
- docs/library/tasks/B-TASK-09.md
- docs/library/tasks/B-TASK-10.md
- docs/library/skills/S-LEARN-01.md
- docs/library/skills/S-LEARN-02.md
- docs/library/skills/S-LEARN-03.md
- docs/library/skills/S-LEARN-04.md
- docs/library/skills/S-LEARN-05.md
- docs/library/skills/S-LEARN-06.md
- docs/library/skills/S-LEARN-07.md
- docs/library/skills/S-LEARN-08.md
- docs/library/SKILLS.md

- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([... ], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

---

# 3.4 Execution Plan (A-E Canon)

## 3.4.1 State

- **Context:** The Core is hardened (Phase 3), but the Library Trinity (Agents, Tasks, Skills) operates on generic assumptions ("Citizen Model") rather than mechanistic tethering ("Federation Protocol").
- **Drift:** Agents lack explicit instructions to read STELA.md, tasks rely on implicit handoffs, and legacy skills lack provenance or stack-agnostic logic. Contractions exist in agent prompts, violating PoT.md linguistic standards.
- **Desired State:** A fully federated Library where agents are legally bound to STELA.md, tasks explicitly chain resources, and skills are robust against context hazards.

## 3.4.2 Request

**Numbered tasks (concrete, no vibes):**

1. **Agent Refactoring (R-AGENT-01..06):**
   - Inject the "Jurisdiction Preamble" at the very top of each file:
     JURISDICTION: Federation.
     AUTHORITY: Derived from STELA.md.
     CONSTRAINT: Cannot alter Core (ops/, docs/, tools/).

PRIMARY DIRECTIVE: Align all output with the local project constitution (STELA.md). If STELA.md is silent, default to Platform Core standards.
    - Remove ALL contractions in the file text (e.g., "don't" -> "do not", "can't" -> "cannot", "you're" -> "you are").
    - Ensure tools and models are defined in the frontmatter.
2. **Task Refactoring (B-TASK-01..10):**
    - Add a "Dependencies" section to each task listing required Agents and Skills by ID.
    - Replace any legacy terms ("Sheriff", "Colonies") with "Governance Engine" and "Federation".
    - Ensure the task explicitly instructs the agent to verify the STELA.md of the target project.
3. **Skill Refactoring (S-LEARN-01..08):**
    - Ensure every skill file has a ## Provenance block. If missing, add a block marked "Captured: Legacy / Backfilled".
    - Inject "Context Hazard" warning at the top of every skill: "Context Hazard: This skill is for production payloads only. Do not add to global context."
    - Refactor S-LEARN-02 to be stack-agnostic (wrap Next.js/FastAPI rules in "IF Stack = Stela Standard" logic).
4. **Template Update:**
    - Update docs/library/SKILLS.md to include the Context Hazard warning in the Promotion Packet template.

## 3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
    - Refactored 6 Agents to enforce Federation Protocol and STELA.md tethering.
    - Refactored 10 Tasks to define explicit Agent/Skill dependencies and remove legacy metaphors.
    - Refactored 8 Skills to enforce Context Hazard warnings and provenance backfilling.
    - Removed linguistic contractions across the Library Trinity.

## 3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

## 3.4.5 Receipt (Required)

### Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
    - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
    - Prefer: bash tools/lint/context.sh

    - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
    - Prefer: bash tools/lint/truth.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)

### Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
    - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

### Artifact bundle (required directory)

- storage/handoff/

### Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0027-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

**MANDATE:** No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

**REQUIRED OUTPUT**

**MUST include a "Status" block at the top of RESULTS:**

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

**RESULTS file must include (minimum)**

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

**Storage artifact flow (local-only, untracked):**

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

# 4. Closeout (Mandatory)

## Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0027-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

### Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

### Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

---

## 4.1 Thread Transition (Reset / Archive Rule)

**Purpose:** Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

## 5. Work Log (Timestamped Continuity)

**Rule:** This is not a transcript. It’s the durable breadcrumb trail.

**Expectation:** Append a new timestamped entry on every DP closeout (no exceptions).

**Each entry ends with:** NEXT: <one single action>.

- **Example format:** 2026-01-27 14:05 — DP-OPS-0005: Example entry only. Verification: NOT RUN. Blockers: none. NEXT: follow-up.

---

# 8. Conclusion

The Stela ecosystem has achieved a "Professional Autopoiesis" state. The Core stands solid, the Governance Engine operates actively, and the history remains secure. The implementation of DP-OPS-0026 proved a complete success, creating the necessary constitutional foundation for autonomous scaling.

The "Deep Dive" into the Library serves not just as a maintenance task; it constitutes the Cognitive Awakening of the system. By tethering agents to their specific STELA.md realities, we move from "generic AI assistance" to "specialized, constitutionally-bound federated labor." This alignment represents the final step in ensuring that when the system replicates, it replicates order, not entropy.

**Recommendation:** Proceed immediately with DP-OPS-0027. The system stands ready.

### Works cited

1. Stela AI Constitution_ DP-OPS-0026 Analysis.pdf